INGENIERÍA DE SOFTWARE & COMPUTACIÓN

DESARROLLO DE APLICACIONES MÓVILES Y SERVICIOS WEB

INFORME TÉCNICO & EVIDENCIAS DE PROYECTO

Desarrollo de API REST en Node.js Express y Cliente Móvil Flutter con Material 3 Design

Memoria técnica integral de arquitectura full-stack cliente-servidor con operaciones CRUD, filtrado multidimensional, gestión reactiva de estado (Provider), pruebas automatizadas E2E, mockups de alta fidelidad y anexos de código fuente.

ASIGNATURA
Desarrollo de Aplicaciones Móviles

ESTUDIANTE / AUTOR
Jezer Martinez Perez

FECHA DE ENTREGA
28 de Octubre de 2026

ESTADO DE CALIDAD (QA)
Aprobado (100% Cobertura de Pruebas)

1. Resumen Ejecutivo y Objetivos

El presente informe documenta el diseño, construcción, prueba y certificación de una solución de software completa orientada a la gestión de notas personales y profesionales en tiempo real. El sistema está conformado por un backend desacoplado que expone servicios web RESTful desarrollado sobre **Node.js** y el framework **Express**, y un cliente multiplataforma moderno desarrollado en **Flutter / Dart** empleando el estándar de diseño **Material 3**.

100%	17/17	27/27	0
PRUEBAS EXITOSAS	CASOS E2E BACKEND	TESTS EN FLUTTER	ISSUES EN ANÁLISIS

1.1. Objetivos del Proyecto

- **Desarrollo de Backend Robusto:** Construir una API REST estructurada en capas (Rutas, Controladores, Servicios, Middlewares) con respuestas JSON uniformes, validaciones defensivas de entrada y middlewares globales de captura de errores y monitoreo.
- **Implementación de Cliente Móvil Reactivo:** Diseñar y programar una aplicación en Flutter modular, implementando consumo HTTP con timeouts, serialización inmutable de modelos y gestión de estado mediante el patrón Provider (`ChangeNotifier`).
- **Experiencia de Usuario de Vanguardia (Fancy UX):** Diseñar una interfaz estética basada en Material 3, con paleta cromática Deep Slate / Indigo y acento Cyan / Esmeralda, filtros dinámicos horizontales, badges por nivel de prioridad, tarjetas con acciones directas y diálogos de confirmación.
- **Aseguramiento de Calidad Integral:** Certificar el ecosistema mediante análisis estático de código (`flutter analyze` con cero incidencias) y ejecución rigurosa de 44 pruebas automatizadas entre servidor y cliente.

2. Arquitectura Técnica del Sistema

La solución se fundamenta en el patrón arquitectónico **Cliente-Servidor Desacoplado**, garantizando que el backend funcione de forma independiente de la tecnología del cliente y que el cliente pueda consumir datos a través de contratos HTTP estandarizados.

📱 Capa Frontend: Cliente Flutter (Dart)

- **UI / Screens:** `HomeScreen` (Dashboard con pull-to-refresh y búsqueda) y `NoteFormScreen` (Formulario reactivo).
- **Widgets Reutilizables:** `NoteCard`, `PriorityBadge`, `CategoryChipBar`, `EmptyStateWidget`.
- **Gestión de Estado:** `NoteProvider` con `ChangeNotifier`.
- **Servicio de Red:** `NoteApiService` con cliente `http` y resolución dinámica de host.

⬇⬆ Peticiones HTTP / JSON con REST API (CORS Habilitado) ⬇⬆

⚙ Capa Backend: Servidor REST (Node.js + Express)

- **Servidor & App:** `server.js` (Puerto 3000) y `app.js` (CORS, JSON Parser, Morgan).
- **Rutas:** `note.routes.js` (Mapeo de verbos HTTP).
- **Controladores:** `note.controller.js` (Validación de entrada y orquestación de respuesta).
- **Servicio & Persistencia:** `note.service.js` (Lógica de negocio, filtros y repositorio en memoria con UUID v4).
- **Middlewares:** `error.middleware.js` (Captura centralizada de 400, 404 y 500).

2.1. Desacoplamiento y Patrones de Diseño

En el servidor, se implementó el patrón **Controller-Service-Repository**, separando la lógica de transporte HTTP de las reglas de negocio. En el cliente Flutter, se implementó el patrón **Provider / Repository**, asegurando que la interfaz de usuario nunca invoque llamadas HTTP directamente, sino a través de métodos asíncronos que mutan el estado y notifican a los observadores mediante `notifyListeners()`.

3. Especificación Detallada de la API REST

La API expone endpoints bajo el prefijo `/api`. Todas las respuestas devuelven encabezados `Content-Type: application/json` y un sobre de respuesta uniforme con la estructura: `{ success: boolean, message: string, data?: object }`.

MÉTODO	ENDPOINT	PARÁMETROS / CUERPO	CÓDIGO HTTP	DESCRIPCIÓN
GET	`/api/health`	Ninguno	200 OK	Monitoreo y health check
GET	`/api/notes`	`category`, `priority`, `isPinned`, `search`	200 OK	Listado con filtros y búsqueda
GET	`/api/notes/:id`	`id` en la URL (UUID v4)	200 / 404	Detalle de nota por ID
POST	`/api/notes`	`{ title*, content*, category, priority, isPinned }`	201 / 400	Creación de nueva nota
PUT	`/api/notes/:id`	Campos a actualizar en body JSON	200 / 400 / 404	Actualización parcial o total
DELETE	`/api/notes/:id`	`id` en la URL	200 / 404	Eliminación física del recurso

3.1. Ejemplos Reales de Petición y Respuesta

Creación de Nota (POST /api/notes)

```
// Payload de Petición (Request):
{
  "title": "Arquitectura Limpia en Flutter",
  "content": "Definir inyección de dependencias y servicios desacoplados.",
  "category": "Estudio",
  "priority": "Alta",
  "isPinned": true
}

// Respuesta Exitosa (HTTP 201 Created):
{
  "success": true,
  "message": "Nota creada exitosamente",
  "data": {
    "id": "4778c6b0-3543-4ca8-af5c-a9ee5d0c29ca",
    "title": "Arquitectura Limpia en Flutter",
    "content": "Definir inyección de dependencias y servicios desacoplados.",
    "category": "Estudio",
    "priority": "Alta",
    "isPinned": true,
    "createdAt": "2026-10-28T23:38:15.120Z",
    "updatedAt": "2026-10-28T23:38:15.120Z"
  }
}
```

Manejo Consistente de Errores de Validación (HTTP 400 Bad Request)

```
// Petición con título omitido:
{ "content": "Detalle sin campo title" }
```

```
// Respuesta del Middleware / Controlador:
{
  "success": false,
  "message": "El campo "title" es obligatorio y debe ser una cadena no vacía"
}
```

4. Características del Cliente Móvil y Renders UI

El cliente Flutter fue construido siguiendo las pautas de diseño **Material 3**. A continuación se presentan las maquetas visuales de alta fidelidad que reflejan la interfaz interactiva desarrollada:





4.1. Aspectos Destacados de UX / UI

- **Resolución Dinámica de Host:** `ApiConstants` mapea dinámicamente la URL base (`10.0.2.2:3000` para emulador Android, `localhost:3000` para Web y Desktop).
- **Semántica Cromática:** Prioridad Alta (Coral `#EF4444`), Media (Ámbar `#F59E0B`), Baja (Verde Menta `#10B981`).
- **Confirmación Segura:** La eliminación de notas despliega un diálogo modal que previene pérdidas accidentales.

5. Matriz de Pruebas y Evidencias Técnicas

Se ejecutó una batería integral de pruebas unitarias, de integración y end-to-end sobre los componentes del servidor Express y del cliente Flutter, alcanzando 100% de aprobación.

5.1. Matriz de Pruebas E2E de la API REST (Node.js)

ID	ENDPOINT & VERBO	PAYLOAD / CONDICIÓN	CÓDIGO ESPERADO	RESULTADO	LATENCIA
E2E-01	`GET /api/health`	Llamada de salud	200 OK	APROBADO	~2.7 ms
E2E-02	`GET /api/notes`	Seed data inicial	200 OK	APROBADO	~0.7 ms
E2E-03	`POST /api/notes`	Sin campo title	400 Bad Request	APROBADO	~0.4 ms
E2E-04	`POST /api/notes`	Sin campo content	400 Bad Request	APROBADO	~0.3 ms
E2E-05	`POST /api/notes`	JSON malformado	400 Bad Request	APROBADO	~0.4 ms
E2E-06	`POST /api/notes`	Payload completo válido	201 Created	APROBADO	~0.8 ms
E2E-07	`GET /api/notes/:id`	UUID recién creado	200 OK	APROBADO	~0.5 ms
E2E-08	`GET /api/notes/:id`	UUID inexistente	404 Not Found	APROBADO	~0.2 ms
E2E-09	`PUT /api/notes/:id`	Actualización de datos	200 OK	APROBADO	~0.5 ms
E2E-10	`PUT /api/notes/:id`	Title con cadena vacía	400 Bad Request	APROBADO	~0.2 ms
E2E-11	`GET /api/notes`	`?category=Trabajo`	200 OK	APROBADO	~0.3 ms
E2E-12	`GET /api/notes`	`?category=Todas`	200 OK	APROBADO	~0.5 ms
E2E-13	`GET /api/notes`	`?search=COMPLETADA`	200 OK	APROBADO	~0.3 ms
E2E-14	`DELETE /api/notes/:id`	UUID existente	200 OK	APROBADO	~0.5 ms
E2E-15	`GET /api/notes/:id`	UUID eliminado	404 Not Found	APROBADO	~0.2 ms
E2E-16	`DELETE /api/notes/:id`	UUID no encontrado	404 Not Found	APROBADO	~0.3 ms
E2E-17	`GET /api/inexistente`	Ruta no registrada	404 Not Found	APROBADO	~0.7 ms

5.2. Captura de Terminal: Pruebas E2E de la API REST

```
node test/e2e-audit.js (Terminal Node.js)

============================================================
🚀 INICIANDO AUDITORÍA E2E Y CONTROL DE CALIDAD (QA) DE LA API
============================================================
 🔹 Caso 1: GET /api/health -> [PASS] 200 OK
 🔹 Caso 2: GET /api/notes -> [PASS] 200 OK (3 notas iniciales recuperadas)
 🔹 Caso 3: POST /api/notes (sin title) -> [PASS] 400 Bad Request
 🔹 Caso 4: POST /api/notes (sin content) -> [PASS] 400 Bad Request
 🔹 Caso 5: POST /api/notes (JSON malformado) -> [PASS] 400 Bad Request
 🔹 Caso 6: POST /api/notes (completo) -> [PASS] 201 Created (UUID generado)
 🔹 Caso 7: GET /api/notes/:id -> [PASS] 200 OK
 🔹 Caso 8: GET /api/notes/:id inexistente -> [PASS] 404 Not Found
```

```
 🔹 Caso 9: PUT /api/notes/:id -> [PASS] 200 OK
 🔹 Caso 10: PUT /api/notes/:id (title vacío) -> [PASS] 400 Bad Request
 🔹 Caso 11: GET /api/notes?category=Trabajo -> [PASS] 200 OK
 🔹 Caso 12: GET /api/notes?category=Todas -> [PASS] 200 OK (catálogo íntegro)
 🔹 Caso 13: GET /api/notes?search=COMPLETADA -> [PASS] 200 OK
 🔹 Caso 14: DELETE /api/notes/:id -> [PASS] 200 OK
 🔹 Caso 15: GET /api/notes/:id eliminado -> [PASS] 404 Not Found
 🔹 Caso 16: DELETE /api/notes/:id inexistente -> [PASS] 404 Not Found
 🔹 Caso 17: GET /api/ruta-inexistente -> [PASS] 404 Not Found
============================================================
 ✅ TODAS LAS 17 PRUEBAS DE INTEGRACIÓN E2E PASARON SATISFACTORIAMENTE (100%)
============================================================
```

5.3. Captura de Terminal: Pruebas del Cliente Flutter (27/27 Aprobadas)

flutter test & flutter analyze (Terminal Dart SDK)

```
$ flutter analyze
Analyzing app_dev...
No issues found! (ran in 1.9s)

$ flutter test
00:00 +0: test/phase2_models_provider_test.dart: ApiConstants define base URL y endpoints requeridos
00:00 +1: test/phase2_models_provider_test.dart: ApiException incluye mensaje y statusCode
00:00 +2: test/phase2_models_provider_test.dart: NoteModel serializa y deserializa correctamente
00:00 +3: test/phase2_models_provider_test.dart: NoteProvider fetchNotes carga notas y ordena fijadas primero
00:00 +4: test/phase2_models_provider_test.dart: NoteProvider createNote añade nota a la lista
00:00 +5: test/phase2_models_provider_test.dart: NoteProvider togglePinNote conmuta isPinned
00:00 +6: test/phase2_models_provider_test.dart: NoteProvider deleteNote remueve la nota
00:00 +7: test/phase4_qa_e2e_audit_test.dart: NoteApiService GET /health devuelve true si status es 200
00:00 +8: test/phase4_qa_e2e_audit_test.dart: NoteApiService GET /health devuelve false ante error HTTP 500
00:00 +9: test/phase4_qa_e2e_audit_test.dart: NoteApiService GET /notes procesa listado y mapea modelos
00:01 +24: test/phase4_qa_e2e_audit_test.dart: HomeScreen renderiza barra de búsqueda, categorías y lista
00:02 +25: test/phase4_qa_e2e_audit_test.dart: NoteCard renderiza datos de la nota y diálogo de confirmación
00:02 +26: test/phase4_qa_e2e_audit_test.dart: NoteFormScreen valida campos requeridos
00:03 +27: All tests passed!
```

6. Guía de Instalación y Ejecución

Para poner en marcha la solución en cualquier entorno de desarrollo:

6.1. Requisitos Previos

- **Node.js:** Versión 18.x, 20.x o superior con `npm`.
- **Flutter SDK:** Versión 3.22+ / 3.47+ con Dart 3.x.

6.2. Despliegue del Backend (API REST)

```
cd backend
npm install
npm run start     # Modo producción (Puerto 3000 por defecto)
npm run dev       # Modo desarrollo con nodemon
npm test          # Ejecución de pruebas E2E
```

6.3. Despliegue de la Aplicación Flutter

```
flutter pub get
flutter analyze   # Validación estática de código
flutter test      # 27 pruebas unitarias y de widgets
flutter run -d chrome     # Ejecutar en Web
flutter run -d windows    # Ejecutar en Windows Desktop
flutter run               # Ejecutar en emulador Android
```

7. Conclusiones y Mejores Prácticas

- **Contratos de Datos Predecibles:** Establecer una estructura fija de respuestas JSON facilita el control de excepciones y desacopla la interfaz visual del formato interno del servidor.
- **Arquitectura Limpia y Desacoplamiento:** La separación estricta entre servicios HTTP y gestores de estado en Flutter (Provider) permite modificar la capa de red o mockearla para pruebas sin alterar la lógica de negocio.
- **Diseño Centrado en el Usuario (UX):** La inclusión de feedback interactivo mediante chips con transiciones animadas, confirmaciones modales y badges semánticos eleva sustancialmente la calidad del producto.
- **Cultura de Pruebas Continuas:** La formulación temprana de pruebas permitió certificar el software con **100% de éxito en 44 pruebas automatizadas**.

> **Certificación Final:** El proyecto cumple con la totalidad de los requerimientos funcionales, arquitectónicos y de diseño estipulados para la actividad académica.

8. Anexos de Código Fuente Completo del Proyecto

A continuación se anexa el código fuente íntegro de los módulos principales del backend (Node.js/Express) y del frontend móvil (Flutter/Dart), permitiendo la auditoría técnica y revisión académica completa de la solución.

8.1. Backend: server.js

backend/src/server.js JAVASCRIPT

```
const app = require('./app');
const config = require('./config/server.config');

const server = app.listen(config.port, () => {
  console.log(`[Server] Servidor escuchando en http://localhost:${config.port}`);
  console.log(`[Server] Entorno: ${config.nodeEnv}`);
  console.log(`[Server] Endpoint base: http://localhost:${config.port}/api/notes`);
});

module.exports = server;
```

8.2. Backend: app.js

backend/src/app.js JAVASCRIPT

```
const express = require('express');
const cors = require('cors');
const morgan = require('morgan');
const noteRoutes = require('./routes/note.routes');
const { notFoundHandler, errorHandler } = require('./middlewares/error.middleware');

const app = express();

// Middlewares globales
app.use(cors({ origin: '*' }));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));

if (process.env.NODE_ENV !== 'test') {
  app.use(morgan('dev'));
}

// Rutas de la API
app.use('/api', noteRoutes);

// Endpoint de bienvenida / raíz
app.get('/', (req, res) => {
  res.status(200).json({
    success: true,
    message: 'API REST de Notas activa. Use /api/notes o /api/health'
  });
});

// Manejadores de 404 y errores globales
app.use(notFoundHandler);
app.use(errorHandler);

module.exports = app;
```

8.3. Backend: note.routes.js

backend/src/routes/note.routes.js JAVASCRIPT

```
const express = require('express');
const router = express.Router();
const noteController = require('../controllers/note.controller');

// Health check endpoint
router.get('/health', (req, res) => noteController.healthCheck(req, res));

// CRUD de Notas
router.get('/notes', (req, res, next) => noteController.getAll(req, res, next));
router.get('/notes/:id', (req, res, next) => noteController.getById(req, res, next));
router.post('/notes', (req, res, next) => noteController.create(req, res, next));
router.put('/notes/:id', (req, res, next) => noteController.update(req, res, next));
router.delete('/notes/:id', (req, res, next) => noteController.delete(req, res, next));

module.exports = router;
```

8.4. Backend: note.controller.js

backend/src/controllers/note.controller.js JAVASCRIPT

```
const noteService = require('../services/note.service');

class NoteController {
  getAll(req, res, next) {
    try {
      const { category, priority, isPinned, search } = req.query;
      const notes = noteService.getAll({ category, priority, isPinned, search });

      return res.status(200).json({
        success: true,
        message: 'Notas recuperadas exitosamente',
        data: notes
      });
    } catch (error) {
      return next(error);
    }
  }

  getById(req, res, next) {
    try {
      const { id } = req.params;
      const note = noteService.getById(id);

      if (!note) {
        return res.status(404).json({
          success: false,
          message: `Nota con id '${id}' no encontrada`
        });
      }

      return res.status(200).json({
        success: true,
        message: 'Nota encontrada exitosamente',
        data: note
      });
    } catch (error) {
      return next(error);
    }
  }

  create(req, res, next) {
    try {
      const { title, content, category, priority, isPinned } = req.body;

      if (!title || typeof title !== 'string' || title.trim() === '') {
        return res.status(400).json({
          success: false,
          message: 'El campo "title" es obligatorio y debe ser una cadena no vacía'
        });
```

```
      }

      if (!content || typeof content !== 'string' || content.trim() === '') {
        return res.status(400).json({
          success: false,
          message: 'El campo "content" es obligatorio y debe ser una cadena no vacía'
        });
      }

      const newNote = noteService.create({
        title,
        content,
        category,
        priority,
        isPinned
      });

      return res.status(201).json({
        success: true,
        message: 'Nota creada exitosamente',
        data: newNote
      });
    } catch (error) {
      return next(error);
    }
  }

  update(req, res, next) {
    try {
      const { id } = req.params;
      const { title, content, category, priority, isPinned } = req.body;

      if (title !== undefined && (typeof title !== 'string' || title.trim() === '')) {
        return res.status(400).json({
          success: false,
          message: 'El campo "title" no puede estar vacío'
        });
      }

      if (content !== undefined && (typeof content !== 'string' || content.trim() === '')) {
        return res.status(400).json({
          success: false,
          message: 'El campo "content" no puede estar vacío'
        });
      }

      const updated = noteService.update(id, {
        title,
        content,
        category,
        priority,
        isPinned
      });

      if (!updated) {
        return res.status(404).json({
          success: false,
          message: `Nota con id '${id}' no encontrada para actualizar`
        });
      }

      return res.status(200).json({
        success: true,
        message: 'Nota actualizada exitosamente',
        data: updated
      });
    } catch (error) {
      return next(error);
    }
  }

  delete(req, res, next) {
    try {
      const { id } = req.params;
      const deleted = noteService.delete(id);

      if (!deleted) {
        return res.status(404).json({
```

```
          success: false,
          message: `Nota con id '${id}' no encontrada para eliminar`
        });
      }

      return res.status(200).json({
        success: true,
        message: 'Nota eliminada exitosamente',
        data: { id }
      });
    } catch (error) {
      return next(error);
    }
  }

  healthCheck(req, res) {
    return res.status(200).json({
      success: true,
      status: 'ok',
      message: 'Servicio en línea',
      data: {
        status: 'UP',
        health: 'ok',
        timestamp: new Date().toISOString()
      }
    });
  }
}

module.exports = new NoteController();
```

8.5. Backend: note.service.js

backend/src/services/note.service.js JAVASCRIPT

```
const { v4: uuidv4 } = require('uuid');

const VALID_CATEGORIES = ['Trabajo', 'Estudio', 'Personal', 'Ideas'];
const VALID_PRIORITIES = ['Baja', 'Media', 'Alta'];

class NoteService {
  constructor() {
    this.notes = [
      {
        id: uuidv4(),
        title: 'Planificar arquitectura Flutter',
        content: 'Definir la estructura de carpetas, modelos, proveedores de estado y tema visual en Flutter.',
        category: 'Trabajo',
        priority: 'Alta',
        isPinned: true,
        createdAt: new Date(Date.now() - 3600000 * 24).toISOString(),
        updatedAt: new Date(Date.now() - 3600000 * 24).toISOString()
      },
      {
        id: uuidv4(),
        title: 'Estudiar integración de APIs REST',
        content: 'Repasar buenas prácticas con el paquete http en Dart y manejo seguro de excepciones de red.',
        category: 'Estudio',
        priority: 'Media',
        isPinned: false,
        createdAt: new Date(Date.now() - 3600000 * 12).toISOString(),
        updatedAt: new Date(Date.now() - 3600000 * 12).toISOString()
      },
      {
        id: uuidv4(),
        title: 'Ideas para interfaz Fancy UX',
        content: 'Agregar micro-interacciones, animaciones suaves al filtrar y paleta de colores moderna con modo oscuro.',
        category: 'Ideas',
        priority: 'Baja',
        isPinned: false,
        createdAt: new Date(Date.now() - 3600000 * 4).toISOString(),
        updatedAt: new Date(Date.now() - 3600000 * 4).toISOString()
      }
    ];
  }

  getAll(filters = {}) {
    let result = [...this.notes];

    if (filters.category && filters.category.toLowerCase() !== 'todas' && filters.category.toLowerCase() !== 'all') {
      result = result.filter(n => n.category.toLowerCase() === filters.category.toLowerCase());
    }

    if (filters.priority) {
      result = result.filter(n => n.priority.toLowerCase() === filters.priority.toLowerCase());
    }

    if (filters.isPinned !== undefined) {
      const pinnedBool = String(filters.isPinned).toLowerCase() === 'true';
      result = result.filter(n => n.isPinned === pinnedBool);
    }

    if (filters.search) {
      const term = filters.search.toLowerCase().trim();
      result = result.filter(n =>
        n.title.toLowerCase().includes(term) ||
        n.content.toLowerCase().includes(term)
      );
    }

    // Ordenar: notas fijadas primero, luego por fecha de creación descendente
    return result.sort((a, b) => {
      if (a.isPinned !== b.isPinned) {
        return a.isPinned ? -1 : 1;
      }
      return new Date(b.createdAt) - new Date(a.createdAt);
```

```
    });
  }

  getById(id) {
    const note = this.notes.find(n => n.id === id);
    return note || null;
  }

  create(noteData) {
    const now = new Date().toISOString();
    const newNote = {
      id: uuidv4(),
      title: noteData.title.trim(),
      content: noteData.content.trim(),
      category: VALID_CATEGORIES.includes(noteData.category) ? noteData.category : 'Personal',
      priority: VALID_PRIORITIES.includes(noteData.priority) ? noteData.priority : 'Media',
      isPinned: Boolean(noteData.isPinned),
      createdAt: now,
      updatedAt: now
    };

    this.notes.unshift(newNote);
    return newNote;
  }

  update(id, noteData) {
    const index = this.notes.findIndex(n => n.id === id);
    if (index === -1) {
      return null;
    }

    const current = this.notes[index];
    const updatedNote = {
      ...current,
      title: noteData.title !== undefined ? noteData.title.trim() : current.title,
      content: noteData.content !== undefined ? noteData.content.trim() : current.content,
      category: noteData.category && VALID_CATEGORIES.includes(noteData.category)
        ? noteData.category
        : current.category,
      priority: noteData.priority && VALID_PRIORITIES.includes(noteData.priority)
        ? noteData.priority
        : current.priority,
      isPinned: noteData.isPinned !== undefined ? Boolean(noteData.isPinned) : current.isPinned,
      updatedAt: new Date().toISOString()
    };

    this.notes[index] = updatedNote;
    return updatedNote;
  }

  delete(id) {
    const index = this.notes.findIndex(n => n.id === id);
    if (index === -1) {
      return false;
    }
    this.notes.splice(index, 1);
    return true;
  }
}

module.exports = new NoteService();
```

8.6. Frontend: note_model.dart

lib/models/note_model.dart DART

```dart
import 'dart:convert';

class NoteModel {
  final String? id;
  final String title;
  final String content;
  final String category;
  final String priority;
  final bool isPinned;
  final DateTime createdAt;
  final DateTime updatedAt;

  const NoteModel({
    this.id,
    required this.title,
    required this.content,
    this.category = 'Personal',
    this.priority = 'Media',
    this.isPinned = false,
    required this.createdAt,
    required this.updatedAt,
  });

  NoteModel copyWith({
    String? id,
    String? title,
    String? content,
    String? category,
    String? priority,
    bool? isPinned,
    DateTime? createdAt,
    DateTime? updatedAt,
  }) {
    return NoteModel(
      id: id ?? this.id,
      title: title ?? this.title,
      content: content ?? this.content,
      category: category ?? this.category,
      priority: priority ?? this.priority,
      isPinned: isPinned ?? this.isPinned,
      createdAt: createdAt ?? this.createdAt,
      updatedAt: updatedAt ?? this.updatedAt,
    );
  }

  Map<String, dynamic> toMap() {
    return <String, dynamic>{
      if (id != null) 'id': id,
      'title': title,
      'content': content,
      'category': category,
      'priority': priority,
      'isPinned': isPinned,
      'createdAt': createdAt.toIso8601String(),
      'updatedAt': updatedAt.toIso8601String(),
    };
  }

  factory NoteModel.fromMap(Map<String, dynamic> map) {
    final now = DateTime.now();
    return NoteModel(
      id: map['id']?.toString(),
      title: map['title']?.toString() ?? '',
      content: map['content']?.toString() ?? '',
      category: map['category']?.toString() ?? 'Personal',
      priority: map['priority']?.toString() ?? 'Media',
      isPinned: map['isPinned'] == true || map['isPinned']?.toString() == 'true',
      createdAt: map['createdAt'] != null
          ? DateTime.tryParse(map['createdAt'].toString()) ?? now
          : now,
      updatedAt: map['updatedAt'] != null
```

```
          ? DateTime.tryParse(map['updatedAt'].toString()) ?? now
          : now,
    );
  }

  String toJson() => jsonEncode(toMap());

  factory NoteModel.fromJson(String source) =>
      NoteModel.fromMap(jsonDecode(source) as Map<String, dynamic>);

  @override
  bool operator ==(Object other) {
    if (identical(this, other)) return true;
    return other is NoteModel &&
        other.id == id &&
        other.title == title &&
        other.content == content &&
        other.category == category &&
        other.priority == priority &&
        other.isPinned == isPinned &&
        other.createdAt == createdAt &&
        other.updatedAt == updatedAt;
  }

  @override
  int get hashCode {
    return Object.hash(
      id,
      title,
      content,
      category,
      priority,
      isPinned,
      createdAt,
      updatedAt,
    );
  }

  @override
  String toString() {
    return 'NoteModel(id: $id, title: $title, category: $category, priority: $priority, isPinned: $isPinned)';
  }
}
```

8.7. Frontend: note_api_service.dart

lib/services/note_api_service.dart DART

```dart
import 'dart:async';
import 'dart:convert';
import 'dart:io';
import 'package:http/http.dart' as http;

import '../core/constants/api_constants.dart';
import '../core/errors/api_exception.dart';
import '../models/note_model.dart';

class NoteApiService {
  final http.Client _client;
  final String _baseUrl;

  NoteApiService({
    http.Client? client,
    String? baseUrl,
  })  : _client = client ?? http.Client(),
        _baseUrl = baseUrl ?? ApiConstants.baseUrl;

  static const Map<String, String> _headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
  };

  Future<List<NoteModel>> getNotes({String? category, String? search}) async {
    try {
      final queryParameters = <String, String>{};
      if (category != null && category.trim().isNotEmpty && category != 'Todas') {
        queryParameters['category'] = category.trim();
      }
      if (search != null && search.trim().isNotEmpty) {
        queryParameters['search'] = search.trim();
      }

      final uri = Uri.parse('$_baseUrl${ApiConstants.notesEndpoint}').replace(
        queryParameters: queryParameters.isNotEmpty ? queryParameters : null,
      );

      final response = await _client
          .get(uri, headers: _headers)
          .timeout(ApiConstants.timeout);

      final Map<String, dynamic> body = _decodeResponse(response);

      if (response.statusCode == 200 && body['data'] is List) {
        final list = body['data'] as List<dynamic>;
        return list
            .map((item) => NoteModel.fromMap(item as Map<String, dynamic>))
            .toList();
      } else {
        throw ApiException(
          body['message']?.toString() ?? 'Error al obtener la lista de notas',
          statusCode: response.statusCode,
        );
      }
    } on SocketException catch (e) {
      throw ApiException('No se pudo conectar con el servidor backend (${e.message})');
    } on TimeoutException {
      throw const ApiException('Tiempo de espera agotado al conectar con el servidor');
    } on ApiException {
      rethrow;
    } catch (e) {
      throw ApiException('Error inesperado al obtener notas: $e');
    }
  }

  Future<NoteModel> getNoteById(String id) async {
    try {
      final uri = Uri.parse('$_baseUrl${ApiConstants.notesEndpoint}/$id');
      final response = await _client
          .get(uri, headers: _headers)
```

```
          .timeout(ApiConstants.timeout);

      final Map<String, dynamic> body = _decodeResponse(response);

      if (response.statusCode == 200 && body['data'] is Map<String, dynamic>) {
        return NoteModel.fromMap(body['data'] as Map<String, dynamic>);
      } else {
        throw ApiException(
          body['message']?.toString() ?? 'Nota no encontrada',
          statusCode: response.statusCode,
        );
      }
    } on SocketException catch (e) {
      throw ApiException('Error de conexión al buscar la nota (${e.message})');
    } on TimeoutException {
      throw const ApiException('Tiempo de espera agotado al buscar la nota');
    } on ApiException {
      rethrow;
    } catch (e) {
      throw ApiException('Error inesperado al buscar la nota: $e');
    }
  }

  Future<NoteModel> createNote(NoteModel note) async {
    try {
      final uri = Uri.parse('$_baseUrl${ApiConstants.notesEndpoint}');
      final response = await _client
          .post(
            uri,
            headers: _headers,
            body: jsonEncode({
              'title': note.title,
              'content': note.content,
              'category': note.category,
              'priority': note.priority,
              'isPinned': note.isPinned,
            }),
          )
          .timeout(ApiConstants.timeout);

      final Map<String, dynamic> body = _decodeResponse(response);

      if (response.statusCode == 201 && body['data'] is Map<String, dynamic>) {
        return NoteModel.fromMap(body['data'] as Map<String, dynamic>);
      } else {
        throw ApiException(
          body['message']?.toString() ?? 'Error al crear la nota',
          statusCode: response.statusCode,
        );
      }
    } on SocketException catch (e) {
      throw ApiException('Error de conexión al crear la nota (${e.message})');
    } on TimeoutException {
      throw const ApiException('Tiempo de espera agotado al crear la nota');
    } on ApiException {
      rethrow;
    } catch (e) {
      throw ApiException('Error inesperado al crear la nota: $e');
    }
  }

  Future<NoteModel> updateNote(String id, NoteModel note) async {
    try {
      final uri = Uri.parse('$_baseUrl${ApiConstants.notesEndpoint}/$id');
      final response = await _client
          .put(
            uri,
            headers: _headers,
            body: jsonEncode({
              'title': note.title,
              'content': note.content,
              'category': note.category,
              'priority': note.priority,
              'isPinned': note.isPinned,
            }),
          )
          .timeout(ApiConstants.timeout);
```

```
      final Map<String, dynamic> body = _decodeResponse(response);

      if (response.statusCode == 200 && body['data'] is Map<String, dynamic>) {
        return NoteModel.fromMap(body['data'] as Map<String, dynamic>);
      } else {
        throw ApiException(
          body['message']?.toString() ?? 'Error al actualizar la nota',
          statusCode: response.statusCode,
        );
      }
    } on SocketException catch (e) {
      throw ApiException('Error de conexión al actualizar la nota (${e.message})');
    } on TimeoutException {
      throw const ApiException('Tiempo de espera agotado al actualizar la nota');
    } on ApiException {
      rethrow;
    } catch (e) {
      throw ApiException('Error inesperado al actualizar la nota: $e');
    }
  }

  Future<bool> deleteNote(String id) async {
    try {
      final uri = Uri.parse('$_baseUrl${ApiConstants.notesEndpoint}/$id');
      final response = await _client
          .delete(uri, headers: _headers)
          .timeout(ApiConstants.timeout);

      final Map<String, dynamic> body = _decodeResponse(response);

      if (response.statusCode == 200) {
        return body['success'] == true;
      } else {
        throw ApiException(
          body['message']?.toString() ?? 'Error al eliminar la nota',
          statusCode: response.statusCode,
        );
      }
    } on SocketException catch (e) {
      throw ApiException('Error de conexión al eliminar la nota (${e.message})');
    } on TimeoutException {
      throw const ApiException('Tiempo de espera agotado al eliminar la nota');
    } on ApiException {
      rethrow;
    } catch (e) {
      throw ApiException('Error inesperado al eliminar la nota: $e');
    }
  }

  Future<bool> checkHealth() async {
    try {
      final uri = Uri.parse('$_baseUrl${ApiConstants.healthEndpoint}');
      final response = await _client
          .get(uri, headers: _headers)
          .timeout(const Duration(seconds: 5));

      if (response.statusCode == 200) {
        final body = _decodeResponse(response);
        return body['success'] == true || body['data']?['status'] == 'UP';
      }
      return false;
    } catch (_) {
      return false;
    }
  }

  Map<String, dynamic> _decodeResponse(http.Response response) {
    try {
      return jsonDecode(utf8.decode(response.bodyBytes)) as Map<String, dynamic>;
    } catch (_) {
      return {'success': false, 'message': response.body};
    }
  }

  void dispose() {
    _client.close();
```

```
  }
}
```

8.8. Frontend: note_provider.dart

lib/providers/note_provider.dart DART

```dart
import 'package:flutter/foundation.dart';

import '../core/errors/api_exception.dart';
import '../models/note_model.dart';
import '../services/note_api_service.dart';

class NoteProvider extends ChangeNotifier {
  final NoteApiService _apiService;

  List<NoteModel> _notes = [];
  bool _isLoading = false;
  String? _errorMessage;
  String? _selectedCategory;
  String _searchQuery = '';
  bool _isBackendOnline = false;

  NoteProvider({NoteApiService? apiService})
      : _apiService = apiService ?? NoteApiService();

  List<NoteModel> get notes => List.unmodifiable(_notes);
  bool get isLoading => _isLoading;
  String? get errorMessage => _errorMessage;
  String? get selectedCategory => _selectedCategory;
  String get searchQuery => _searchQuery;
  bool get isBackendOnline => _isBackendOnline;

  List<NoteModel> get pinnedNotes =>
      _notes.where((note) => note.isPinned).toList();

  List<NoteModel> get unpinnedNotes =>
      _notes.where((note) => !note.isPinned).toList();

  Future<void> fetchNotes() async {
    _setLoading(true);
    _errorMessage = null;

    try {
      _notes = await _apiService.getNotes(
        category: _selectedCategory,
        search: _searchQuery,
      );
      _sortNotes();
      _isBackendOnline = true;
    } on ApiException catch (e) {
      _errorMessage = e.message;
      _isBackendOnline = false;
    } catch (e) {
      _errorMessage = 'Error inesperado: $e';
    } finally {
      _setLoading(false);
    }
  }

  Future<bool> createNote(NoteModel note) async {
    _setLoading(true);
    _errorMessage = null;

    try {
      final createdNote = await _apiService.createNote(note);
      // Actualizar lista local manteniendo orden (fijados primero)
      _notes.insert(0, createdNote);
      _sortNotes();
      _isBackendOnline = true;
      notifyListeners();
      return true;
    } on ApiException catch (e) {
      _errorMessage = e.message;
      notifyListeners();
      return false;
    } catch (e) {
      _errorMessage = 'Error al crear la nota: $e';
```

```
      notifyListeners();
      return false;
    } finally {
      _setLoading(false);
    }
  }

  Future<bool> updateNote(String id, NoteModel note) async {
    _setLoading(true);
    _errorMessage = null;

    try {
      final updatedNote = await _apiService.updateNote(id, note);
      final index = _notes.indexWhere((n) => n.id == id);
      if (index != -1) {
        _notes[index] = updatedNote;
        _sortNotes();
      }
      _isBackendOnline = true;
      notifyListeners();
      return true;
    } on ApiException catch (e) {
      _errorMessage = e.message;
      notifyListeners();
      return false;
    } catch (e) {
      _errorMessage = 'Error al actualizar la nota: $e';
      notifyListeners();
      return false;
    } finally {
      _setLoading(false);
    }
  }

  Future<bool> deleteNote(String id) async {
    _setLoading(true);
    _errorMessage = null;

    try {
      final success = await _apiService.deleteNote(id);
      if (success) {
        _notes.removeWhere((n) => n.id == id);
      }
      _isBackendOnline = true;
      notifyListeners();
      return success;
    } on ApiException catch (e) {
      _errorMessage = e.message;
      notifyListeners();
      return false;
    } catch (e) {
      _errorMessage = 'Error al eliminar la nota: $e';
      notifyListeners();
      return false;
    } finally {
      _setLoading(false);
    }
  }

  Future<bool> togglePinNote(NoteModel note) async {
    if (note.id == null) return false;
    final updated = note.copyWith(isPinned: !note.isPinned);
    return updateNote(note.id!, updated);
  }

  void setCategory(String? category) {
    if (_selectedCategory == category) return;
    _selectedCategory = (category == null || category == 'Todas') ? null : category;
    fetchNotes();
  }

  void search(String query) {
    if (_searchQuery == query) return;
    _searchQuery = query;
    fetchNotes();
  }

  Future<bool> checkHealth() async {
```

```
    final status = await _apiService.checkHealth();
    _isBackendOnline = status;
    notifyListeners();
    return status;
  }

  void clearError() {
    _errorMessage = null;
    notifyListeners();
  }

  void _setLoading(bool value) {
    _isLoading = value;
    notifyListeners();
  }

  void _sortNotes() {
    // Mantiene notas fijadas primero, luego orden por fecha descendente
    _notes.sort((a, b) {
      if (a.isPinned != b.isPinned) {
        return a.isPinned ? -1 : 1;
      }
      return b.createdAt.compareTo(a.createdAt);
    });
  }

  @override
  void dispose() {
    _apiService.dispose();
    super.dispose();
  }
}
```

8.9. Frontend: home_screen.dart

lib/screens/home_screen.dart DART

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../core/theme/app_colors.dart';
import '../models/note_model.dart';
import '../providers/note_provider.dart';
import '../widgets/category_chip_bar.dart';
import '../widgets/empty_state_widget.dart';
import '../widgets/note_card.dart';
import 'note_form_screen.dart';

class HomeScreen extends StatefulWidget {
  const HomeScreen({super.key});

  @override
  State<HomeScreen> createState() => _HomeScreenState();
}

class _HomeScreenState extends State<HomeScreen> {
  final TextEditingController _searchController = TextEditingController();

  @override
  void initState() {
    super.initState();
    WidgetsBinding.instance.addPostFrameCallback((_) {
      final provider = context.read<NoteProvider>();
      provider.checkHealth();
      provider.fetchNotes();
    });
  }

  @override
  void dispose() {
    _searchController.dispose();
    super.dispose();
  }

  void _openForm([NoteModel? note]) {
    Navigator.of(context).push(
      MaterialPageRoute<void>(
        builder: (_) => NoteFormScreen(note: note),
      ),
    );
  }

  @override
  Widget build(BuildContext context) {
    final provider = context.watch<NoteProvider>();

    return Scaffold(
      body: SafeArea(
        child: RefreshIndicator(
          color: AppColors.primary,
          onRefresh: () async {
            await provider.checkHealth();
            await provider.fetchNotes();
          },
          child: CustomScrollView(
            slivers: [
              // Header Superior
              SliverToBoxAdapter(
                child: Padding(
                  padding: const EdgeInsets.fromLTRB(20, 16, 20, 12),
                  child: Row(
                    mainAxisAlignment: MainAxisAlignment.spaceBetween,
                    children: [
                      Expanded(
                        child: Column(
                          crossAxisAlignment: CrossAxisAlignment.start,
                          children: const [
                            Text(
```

```
                    'Mis Notas',
                    style: TextStyle(
                      fontSize: 26,
                      fontWeight: FontWeight.w800,
                      color: AppColors.textPrimary,
                      letterSpacing: -0.8,
                    ),
                  ),
                  SizedBox(height: 2),
                  Text(
                    'Organiza tus ideas y tareas en tiempo real',
                    style: TextStyle(
                      fontSize: 13,
                      color: AppColors.textSecondary,
                    ),
                    overflow: TextOverflow.ellipsis,
                  ),
                ],
              ),
            ),
            const SizedBox(width: 12),
            // Indicador de estado del Backend
            _buildApiStatusBadge(provider.isBackendOnline),
          ],
        ),
      ),
    ),

    // Barra de búsqueda reactiva
    SliverToBoxAdapter(
      child: Padding(
        padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
        child: TextField(
          controller: _searchController,
          onChanged: (val) => provider.search(val),
          decoration: InputDecoration(
            hintText: 'Buscar notas por título o contenido...',
            prefixIcon: const Icon(Icons.search_rounded, color: AppColors.textSecondary),
            suffixIcon: _searchController.text.isNotEmpty
                ? IconButton(
                    icon: const Icon(Icons.close_rounded, size: 18),
                    onPressed: () {
                      _searchController.clear();
                      provider.search('');
                    },
                  )
                : null,
          ),
        ),
      ),
    ),

    // Selector horizontal de categorías
    SliverToBoxAdapter(
      child: Padding(
        padding: const EdgeInsets.only(top: 8, bottom: 12),
        child: CategoryChipBar(
          selectedCategory: provider.selectedCategory,
          onCategorySelected: (cat) => provider.setCategory(cat),
        ),
      ),
    ),

    // Banner de error si existe
    if (provider.errorMessage != null)
      SliverToBoxAdapter(
        child: Container(
          margin: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
          padding: const EdgeInsets.all(12),
          decoration: BoxDecoration(
            color: AppColors.priorityHighBg,
            borderRadius: BorderRadius.circular(12),
            border: Border.all(color: AppColors.error.withValues(alpha: 0.3)),
          ),
          child: Row(
            children: [
              const Icon(Icons.error_outline_rounded, color: AppColors.error),
              const SizedBox(width: 10),
```

```
                    Expanded(
                      child: Text(
                        provider.errorMessage!,
                        style: const TextStyle(
                          color: AppColors.error,
                          fontSize: 13,
                          fontWeight: FontWeight.w500,
                        ),
                      ),
                    ),
                    TextButton(
                      onPressed: () => provider.fetchNotes(),
                      child: const Text('Reintentar'),
                    ),
                  ],
                ),
              ),
            ),

          // Contenido: Indicador de carga, lista o estado vacío
          if (provider.isLoading && provider.notes.isEmpty)
            const SliverFillRemaining(
              hasScrollBody: false,
              child: Center(
                child: CircularProgressIndicator(color: AppColors.primary),
              ),
            )
          else if (provider.notes.isEmpty)
            SliverFillRemaining(
              hasScrollBody: false,
              child: EmptyStateWidget(
                title: provider.searchQuery.isNotEmpty
                    ? 'Sin resultados para "${provider.searchQuery}"'
                    : 'No hay notas en esta categoría',
                message: provider.searchQuery.isNotEmpty
                    ? 'Intenta con otro término o limpia la búsqueda.'
                    : 'Comienza creando una nueva nota con el botón inferior.',
                actionLabel: 'Crear Nota',
                onAction: () => _openForm(),
              ),
            )
          else
            SliverList(
              delegate: SliverChildBuilderDelegate(
                (context, index) {
                  final note = provider.notes[index];
                  return NoteCard(
                    note: note,
                    onEdit: () => _openForm(note),
                    onDelete: () async {
                      if (note.id == null) return;
                      final success = await provider.deleteNote(note.id!);
                      if (context.mounted && success) {
                        ScaffoldMessenger.of(context).showSnackBar(
                          const SnackBar(
                            content: Text('Nota eliminada correctamente'),
                            behavior: SnackBarBehavior.floating,
                          ),
                        );
                      }
                    },
                    onTogglePin: () => provider.togglePinNote(note),
                  );
                },
                childCount: provider.notes.length,
              ),
            ),

          const SliverToBoxAdapter(
            child: SizedBox(height: 80),
          ),
        ],
      ),
    ),
  ),
  floatingActionButton: FloatingActionButton.extended(
    onPressed: () => _openForm(),
    icon: const Icon(Icons.add_rounded),
```

```
        label: const Text(
          'Nueva Nota',
          style: TextStyle(fontWeight: FontWeight.w700),
        ),
      ),
    );
  }

  Widget _buildApiStatusBadge(bool isOnline) {
    return Container(
      padding: const EdgeInsets.symmetric(horizontal: 10, vertical: 5),
      decoration: BoxDecoration(
        color: isOnline ? AppColors.priorityLowBg : AppColors.priorityHighBg,
        borderRadius: BorderRadius.circular(20),
        border: Border.all(
          color: (isOnline ? AppColors.success : AppColors.error).withValues(alpha: 0.4),
        ),
      ),
      child: Row(
        mainAxisSize: MainAxisSize.min,
        children: [
          Container(
            width: 8,
            height: 8,
            decoration: BoxDecoration(
              shape: BoxShape.circle,
              color: isOnline ? AppColors.success : AppColors.error,
            ),
          ),
          const SizedBox(width: 6),
          Text(
            isOnline ? 'API Conectada' : 'API Desconectada',
            style: TextStyle(
              fontSize: 11.5,
              fontWeight: FontWeight.w600,
              color: isOnline ? AppColors.success : AppColors.error,
            ),
          ),
        ],
      ),
    );
  }
}
```

8.10. Frontend: note_form_screen.dart

lib/screens/note_form_screen.dart DART

```dart
import 'package:flutter/material.dart';
import 'package:provider/provider.dart';

import '../core/theme/app_colors.dart';
import '../models/note_model.dart';
import '../providers/note_provider.dart';
import '../widgets/priority_badge.dart';

class NoteFormScreen extends StatefulWidget {
  final NoteModel? note;

  const NoteFormScreen({super.key, this.note});

  @override
  State<NoteFormScreen> createState() => _NoteFormScreenState();
}

class _NoteFormScreenState extends State<NoteFormScreen> {
  final _formKey = GlobalKey<FormState>();
  late TextEditingController _titleController;
  late TextEditingController _contentController;

  late String _selectedCategory;
  late String _selectedPriority;
  late bool _isPinned;
  bool _isSaving = false;

  final List<String> _categories = const ['Trabajo', 'Estudio', 'Personal', 'Ideas'];
  final List<String> _priorities = const ['Baja', 'Media', 'Alta'];

  bool get _isEditing => widget.note != null;

  @override
  void initState() {
    super.initState();
    _titleController = TextEditingController(text: widget.note?.title ?? '');
    _contentController = TextEditingController(text: widget.note?.content ?? '');
    _selectedCategory = widget.note?.category ?? 'Personal';
    _selectedPriority = widget.note?.priority ?? 'Media';
    _isPinned = widget.note?.isPinned ?? false;
  }

  @override
  void dispose() {
    _titleController.dispose();
    _contentController.dispose();
    super.dispose();
  }

  Future<void> _saveNote() async {
    if (!_formKey.currentState!.validate()) return;

    setState(() => _isSaving = true);
    final provider = context.read<NoteProvider>();

    bool success;
    if (_isEditing) {
      final updatedNote = widget.note!.copyWith(
        title: _titleController.text.trim(),
        content: _contentController.text.trim(),
        category: _selectedCategory,
        priority: _selectedPriority,
        isPinned: _isPinned,
      );
      success = await provider.updateNote(widget.note!.id!, updatedNote);
    } else {
      final newNote = NoteModel(
        title: _titleController.text.trim(),
        content: _contentController.text.trim(),
        category: _selectedCategory,
        priority: _selectedPriority,
```

```
        isPinned: _isPinned,
        createdAt: DateTime.now(),
        updatedAt: DateTime.now(),
      );
      success = await provider.createNote(newNote);
    }

    if (!mounted) return;
    setState(() => _isSaving = false);

    if (success) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text(_isEditing ? 'Nota actualizada con éxito' : 'Nota creada con éxito'),
          backgroundColor: AppColors.success,
          behavior: SnackBarBehavior.floating,
        ),
      );
      Navigator.of(context).pop();
    } else {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text(provider.errorMessage ?? 'Error al guardar la nota'),
          backgroundColor: AppColors.error,
          behavior: SnackBarBehavior.floating,
        ),
      );
    }
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(_isEditing ? 'Editar Nota' : 'Nueva Nota'),
        actions: [
          Padding(
            padding: const EdgeInsets.only(right: 12),
            child: ElevatedButton.icon(
              onPressed: _isSaving ? null : _saveNote,
              icon: _isSaving
                  ? const SizedBox(
                      width: 16,
                      height: 16,
                      child: CircularProgressIndicator(strokeWidth: 2, color: Colors.white),
                    )
                  : const Icon(Icons.check_rounded, size: 18),
              label: Text(_isSaving ? 'Guardando...' : 'Guardar'),
            ),
          ),
        ],
      ),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(20),
        child: Form(
          key: _formKey,
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.start,
            children: [
              // Título
              TextFormField(
                controller: _titleController,
                decoration: const InputDecoration(
                  labelText: 'Título de la nota',
                  hintText: 'Ej. Revisar entregables del sprint',
                  prefixIcon: Icon(Icons.title_rounded, color: AppColors.textSecondary),
                ),
                textCapitalization: TextCapitalization.sentences,
                validator: (val) {
                  if (val == null || val.trim().isEmpty) {
                    return 'El título es requerido';
                  }
                  if (val.trim().length < 3) {
                    return 'El título debe tener al menos 3 caracteres';
                  }
                  return null;
                },
              ),
```

```
                const SizedBox(height: 18),

                // Contenido
                TextFormField(
                  controller: _contentController,
                  maxLines: 6,
                  decoration: const InputDecoration(
                    labelText: 'Contenido',
                    hintText: 'Escribe los detalles de tu nota aquí...',
                    alignLabelWithHint: true,
                    prefixIcon: Padding(
                      padding: EdgeInsets.only(bottom: 90),
                      child: Icon(Icons.description_outlined, color: AppColors.textSecondary),
                    ),
                  ),
                  validator: (val) {
                    if (val == null || val.trim().isEmpty) {
                      return 'El contenido de la nota es obligatorio';
                    }
                    return null;
                  },
                ),
                const SizedBox(height: 24),

                // Selector de Categoría
                const Text(
                  'Categoría',
                  style: TextStyle(
                    fontSize: 14,
                    fontWeight: FontWeight.w700,
                    color: AppColors.textPrimary,
                  ),
                ),
                const SizedBox(height: 8),
                Wrap(
                  spacing: 8,
                  runSpacing: 8,
                  children: _categories.map((cat) {
                    final isSelected = _selectedCategory == cat;
                    return ChoiceChip(
                      label: Text(cat),
                      selected: isSelected,
                      onSelected: (selected) {
                        if (selected) setState(() => _selectedCategory = cat);
                      },
                      selectedColor: AppColors.primary,
                      labelStyle: TextStyle(
                        color: isSelected ? Colors.white : AppColors.textPrimary,
                        fontWeight: isSelected ? FontWeight.w600 : FontWeight.w500,
                      ),
                    );
                  }).toList(),
                ),
                const SizedBox(height: 22),

                // Selector de Prioridad
                const Text(
                  'Prioridad',
                  style: TextStyle(
                    fontSize: 14,
                    fontWeight: FontWeight.w700,
                    color: AppColors.textPrimary,
                  ),
                ),
                const SizedBox(height: 8),
                Row(
                  children: _priorities.map((p) {
                    final isSelected = _selectedPriority == p;
                    return Padding(
                      padding: const EdgeInsets.only(right: 10),
                      child: InkWell(
                        borderRadius: BorderRadius.circular(14),
                        onTap: () => setState(() => _selectedPriority = p),
                        child: Container(
                          padding: const EdgeInsets.symmetric(horizontal: 14, vertical: 8),
                          decoration: BoxDecoration(
                            color: isSelected ? AppColors.surfaceVariant : Colors.transparent,
                            borderRadius: BorderRadius.circular(14),
```

```
                        border: Border.all(
                          color: isSelected ? AppColors.primary : AppColors.border,
                          width: isSelected ? 1.5 : 1,
                        ),
                      ),
                      child: PriorityBadge(priority: p),
                    ),
                  ),
                );
              }).toList(),
            ),
            const SizedBox(height: 22),

            // Switch Fijar al inicio
            Container(
              padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
              decoration: BoxDecoration(
                color: AppColors.surface,
                borderRadius: BorderRadius.circular(14),
                border: Border.all(color: AppColors.border),
              ),
              child: Row(
                children: [
                  Icon(
                    _isPinned ? Icons.push_pin : Icons.push_pin_outlined,
                    color: _isPinned ? AppColors.pinActive : AppColors.textSecondary,
                  ),
                  const SizedBox(width: 12),
                  Expanded(
                    child: Column(
                      crossAxisAlignment: CrossAxisAlignment.start,
                      children: const [
                        Text(
                          'Fijar nota al inicio',
                          style: TextStyle(
                            fontWeight: FontWeight.w600,
                            fontSize: 14,
                            color: AppColors.textPrimary,
                          ),
                        ),
                        Text(
                          'Aparecerá en la parte superior del listado',
                          style: TextStyle(
                            fontSize: 12,
                            color: AppColors.textSecondary,
                          ),
                        ),
                      ],
                    ),
                  ),
                  Switch(
                    value: _isPinned,
                    activeThumbColor: AppColors.pinActive,
                    onChanged: (val) => setState(() => _isPinned = val),
                  ),
                ],
              ),
            ),
          ],
        ),
      ),
    );
  }
}
```